

# Operating Systems Lab 1

Nicholas Holm Charlton, Jacob Enbjärde, Xinjie Zhang

September 24, 2026

## 1 Introduction

This lab assignment focused on building a functioning shell program that is able to take user commands and execute them. The task not only involved implementing a shell that can interpret and execute commands, but also allows for command pipelines as well as input/output (I/O) redirection. The tests provided in the lab allowed for further scrutiny when it comes to the robustness of the code, testing whether Ctrl-D is handled correctly as an EOF exit and making sure no zombie processes are created for example.

## 2 Specification Outline

Throughout the lab, we successfully implemented the required specifications. On top of these requirements, the shell also had to be independent in that calls to `system()` to invoke a bash terminal for example were not allowed. The following is the list of specifications.

- Ctrl-D Handling: Respond to Ctrl-D (EOF) to exit.
- Basic Commands: Execute simple commands like `ls`, `date`, and `who`. It should also recognize the `PATH` environment variable to locate commands.
- Background Execution: Allow running commands in the background, e.g., `sleep 30 &`.
- Piping: Support one or more pipes between commands, e.g., `ls | grep out | wc -w`.
- I/O Redirection: Enable standard input and output redirection to files, e.g., `wc -l < /etc/passwd > accounts`.
- Built-ins: Provide `cd` and `exit` as built-in functions.
- Ctrl-C Handling: Ctrl-C should terminate the current foreground process but not the shell itself. Ctrl-C should not affect background jobs; background processes terminate when receiving SIGINT (but not on Ctrl-C).

- No Zombies: Your shell should not leave any zombie processes behind.

### 3 Timeline

We started the lab with implementing the basic commands, this had to be done first since all the other specifications in some way need the shell to be able to execute basic commands. When we had the basic commands we split up a bit so we could work on the different specifications in parallel. We noticed pretty quick that the piping was the most time consuming part so we put a lot of our resources on implementing that. Meanwhile some of us worked on the pipelining we managed to implement background executions, built ins, Ctrl-D and Ctrl-C handling. Then when we got the piping to work properly we did the final implementation of I/O redirection. And every step of the way we tried to make sure we didn't create any zombie processes.

## 4 Implementation

### 4.1 Ctrl-D Handling

When the user presses CTRL-D, the readline function receives NULL and assigns it to a pointer variable, which is dereferenced in later execution. So we need to handle the NULL pointer to avoid segmentation fault by using `exit(0)`. If we do not handle it, the wild pointer will request to access spaces that is not allocated to the current program, which leads to segmentation fault.

### 4.2 Basic Commands

`execvp()` automatically searches the directories listed in the PATH environment variable to locate and execute the binary of `ls`, `date`, `who` as well as many other basic commands. `execvp()` takes the commands "pgm" field, pointing to the "pgmlist[]" array. The first argument contains the actual command. If the command fails, the function will return with error (-1) and then runs `exit(1)` to kill the child process. If the command succeeds, the succeeding lines do not run.

### 4.3 Background Execution

Background execution allows the parent process to continue before the child process finishes executing. So when the command contains "&", it tells the shell that the command is to be executed in the background which sets the "cmd.background" variable to 1. Our program then checks the "cmd.background" to determine what type of command it is. If the command is a foreground command we call "wait(NULL)" to make the parent wait for the child process to finish. To avoid zombie processes being created, we have a function called "sigchld\_handler()" that terminates all background processes when they are

done executing. When the background execution terminates, a SIGCHLD signal is sent to the shell and the shell will jump to this process upon receiving that signal thanks to the "signal(SIGCHLD, sigchld\_handler)" function. The handler function calls "waitpid()" which removes the zombie process from memory.

```
void sigchld_handler(int sig)
{
    (void) sig;
    while (waitpid(-1, NULL, WNOHANG) > 0) {
    }
}
```

## 4.4 Piping

To handle multiple instructions, multiple pipes need to be created. If there are  $n$  instructions,  $n-1$  pipes are needed. Thus, we created an array named `pipefds` with a length of  $2(n-1)$  to hold the file descriptors for both the read and write ends. Because the parsed instructions are stored in reverse order, we need to process them in reverse order. For each child process, we redirect the read and write ends by `dup2()` except for the first and last commands, which either have no pipe input or no pipe output. For a command at position  $i$ , its pipe ends correspond to elements in `pipefds`. Specifically, the read end is redirected from `pipefds[(i - 1) * 2]`, and the write end is redirected to `pipefds[2 * i + 1]`. After all commands have finished, we close all `pipefds` in both the child processes and the parent process to release the file descriptors. We didn't notice the reverse order list at first, so some errors occurred. Then, we adjusted the redirection order according to the given data structure.

## 4.5 I/O Redirection

In order for the I/O redirection to work, the program has to check whether the command contains any input or output files. After checking for this, the program opens the file as read only, or write only, creating the file if it does not exist, and wiping it if it does, depending on whether it contains "stdin" or "stdout" and stores the output of the "open()" command as an "input\_fd" or "out\_fd" integer. The program then uses "dup2()" to overwrite the standard input (the keyboard) or standard output (the terminal screen) with this new file descriptor and finally closes the original descriptor.

## 4.6 Built-ins

A couple of commands, directly given to us as "cd" and "exit" had to be provided as built-in functions. Upon receiving the command, our code checks whether the command is one of these built-in functions before checking if it is something else by comparing the string of the input to "cd" or "exit". "chdir()" and "exit()"

are the built-in functions that are referred to in the lab manual and these are run when the strings match. Since the folder that needs to be entered is held in the second argument of the "cd" command, we use this as the argument for the "chdir()" command. "exit(0)" runs when "exit" is matched with status code 0.

```
if (strcmp(cmd.pgm->pgmlist[0], "exit") == 0)
{ // Checks if the command's string is equal to "exit"
exit(0);
}
if (strcmp(cmd.pgm->pgmlist[0], "cd") == 0)
{ // Checks if the command's string is "cd"
chdir(cmd.pgm->pgmlist[1]);
continue;
}
```

## 4.7 Ctrl-C Handling

Handling the Ctrl-C input required the use of the built-in "signal()" command. To ensure that the shell would not close on Ctrl-C, we use the "signal()" command to ignore SIGINT signals. These signals are used to interrupt the program and setting it to "SIG\_IGN" will tell the shell to ignore them. However, it is also desirable that foreground child processes are terminated with the Ctrl-C interrupt, and since children inherit the characteristics of their parents, they will also exhibit the "SIG\_IGN" behaviour. To allow foreground processes to actually listen to these signals, we set all child processes to "SIG\_DFL" upon creation so that they will listen to these interrupts. The code then distinguishes between foreground and background processes. If the child process is a background process, its process group is changed, so that Ctrl-C cannot interrupt its execution since Ctrl-C is only applied to the parent's process group. This method lets the child process listen to SIGINT signals regardless of whether they are a foreground or background process but Ctrl-C will only interrupt foreground children. We arrived at this particular solution after the specifications had been updated to ensure this is the case. This was a challenge that had arisen after we already implemented a solution that would just set SIGINT to "SIG\_IGN" to background children which was made redundant. We were stuck on this for a bit because we were trying to solve it using just the signal() command, however, the solution required the elegant integration of "setpgid()".

## 5 Feedback

We don't have any concrete feedback to improve the lab. We all thought that it was a good introduction lab to operating systems. The difficulty of the lab was fair but challenging. Regarding the automated tests, we found them very useful and considering the specifications we had to meet we haven't found any missing test cases.